Big Databases : NoSQL

–

Before all :

let's introduce ourselves

- **Me** :
  - Training
  - Work experience
- **You** :
  - Initial training
  - Work/internship experience
  - Level of knowledge in databases, NoSQL, Big data, docker
  - Expectations for this course

Objectives

- 1 : Conceptual understanding of NoSQL databases
- 2 : Proficiency in setting up experimental environment for big databases
- 3 : Master Core NoSQL Databases
- 4 : NoSQL databases integration and choosing the right tool
- 5 : Implement Real-World Data Solutions

Objective 1 : Conceptual understanding of NoSQL databases

- The evolution of data
- Definition of big data
- Traditional databases (RDBMS) limitations
- Introduction to NoSQL Databases

Objective 2 : Proficiency in setting up experimental environment for big databases

- Introduction to containerization
- Docker for Database Environments

Objective 3 : Master Core NoSQL Databases

- Key-Value Stores
- Document Stores
- Column-Family Stores
- Graph Databases

Objective 4 : NoSQL databases integration and choosing the right tool

- When to use each type of NoSQL database?
- Data ingestion strategies (ETL/ELT)
- Introduction to Spark & NoSQL databases

Objective 5 : Implement Real-World Data Solutions

- Presentation and implementation of a global use case

Evaluation methods

–

Evaluations

- A series of application exercises to be completed after lessons no. 3 and 7
- Participation grade
- Final project : last day

My rules

=> Avoid Chatter : talk about the course, ask questions!
=> No wrong question/answer
=> Give your thoughts, share your knowledge
=> **DO NOT USE CHATGPT** or any other AI => try by yourself first : penalties for those who use it

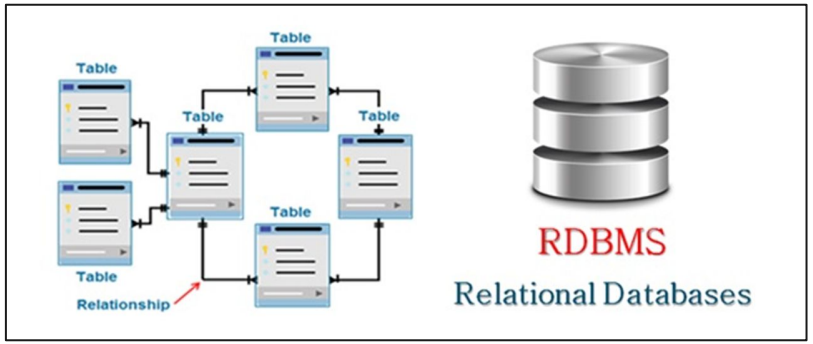
Part 1 :

Big Data Concepts and Challenges

The evolution of data: from structured to big data

Structured data

- Organized in tables (rows and columns)
- Stored in relational databases
- Data managed with SQL in relational DBMSs
- Examples of DBMS :
  - MySQL
  - PostgreSQL
  - Oracle



The evolution of data: from structured to big data

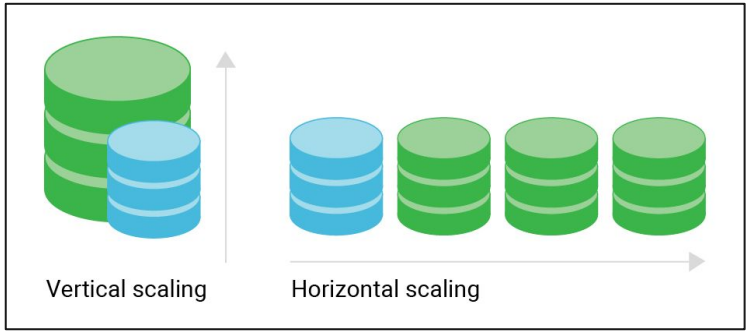
Structured data

- Advantages of relational DBMS :
  - Enforces strong schema (data types, constraints)
    - Types : String, Integer, Boolean, Date, …
    - Relations : PK, Unique, Non NULL, FK, …
  - Easy to query with SQL
  - ACID Transactions :
    - **Atomicity** : A transaction must be fully completed or not executed at all; **All or Nothing**
    - **Consistency** : Transactions must bring the DB from one valid state to another; **Always Valid State**
    - **Isolation** : Multiple transactions can occur simultaneously without interfering with each other; **No Cross-Talk**
    - **Durability** : Once a transaction is committed, it will persist even if there's a crash; **Persistence Guaranteed**

The evolution of data: from structured to big data

Structured data

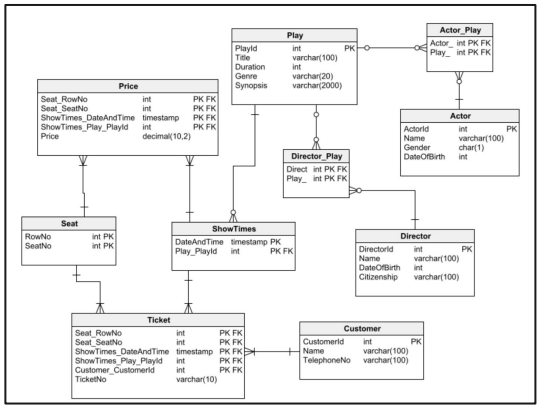
- Limitations :
  - **Scalability** : only optimized for vertical scalability (adding more power to a single server) and not for horizontal scalability (adding more servers)
    => No possibility to use “sharding” (splitting data across servers)



The evolution of data: from structured to big data

Structured data

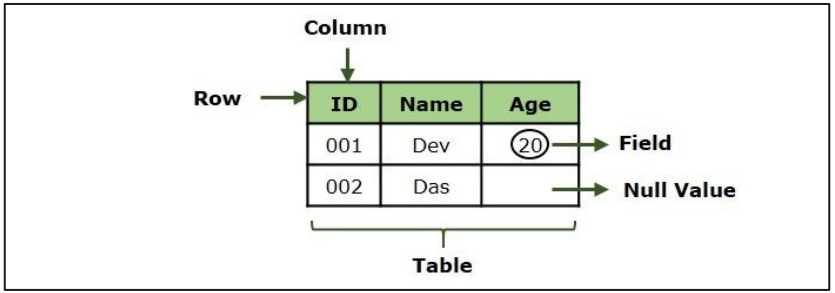
- Limitations :
  - **Schema rigidity** : Adding or modifying columns in production often requires downtime or complex migration scripts
    - Constraints : type, relations …



The evolution of data: from structured to big data

Structured data

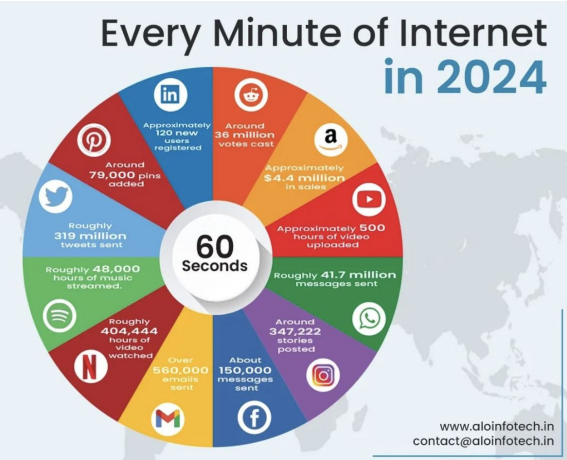
- Limitations :
  - **Inflexibility** : Not optimized for semi-structured, unstructured, hierarchical or high-velocity (real-time) data
    - Images, Audio, Documents, Videos, JSON…

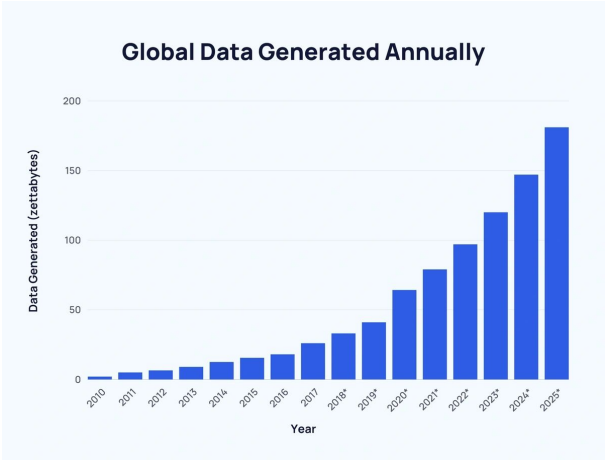


The evolution of data: from structured to big data

Big Data Era

- Explosion of data from:
  - **Social media**





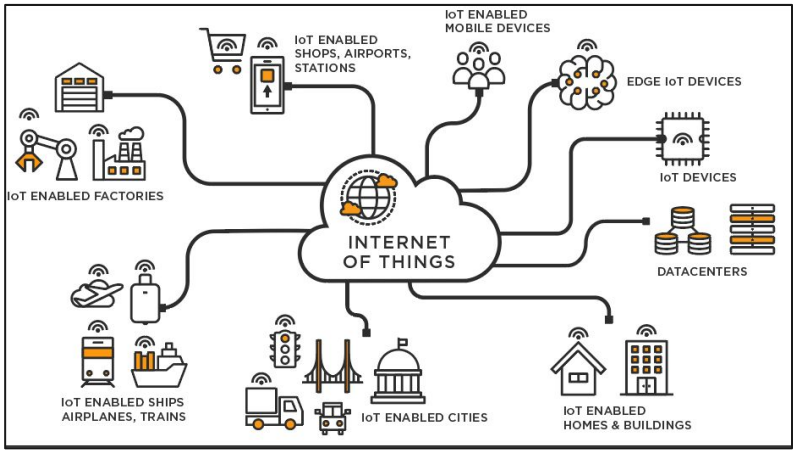
Multiples of bytes				
SI decimal prefixes		Binary usage	IEC binary prefixes	
Name (Symbol)	Value		Name (Symbol)	Value
kilobyte (kB)	10^3	2^{10}	kibibyte (KiB)	2^{10}
megabyte (MB)	10^6	2^{20}	mebibyte (MiB)	2^{20}
gigabyte (GB)	10^9	2^{30}	gibibyte (GiB)	2^{30}
terabyte (TB)	10^{12}	2^{40}	tebibyte (TiB)	2^{40}
petabyte (PB)	10^{15}	2^{50}	pebibyte (PiB)	2^{50}
exabyte (EB)	10^{18}	2^{60}	exbibyte (EiB)	2^{60}
zettabyte (ZB)	10^{21}	2^{70}	zebibyte (ZiB)	2^{70}
yottabyte (YB)	10^{24}	2^{80}	yobibyte (YiB)	2^{80}

See also: Multiples of bits · Orders of magnitude of data

The evolution of data: from structured to big data

Big Data Era

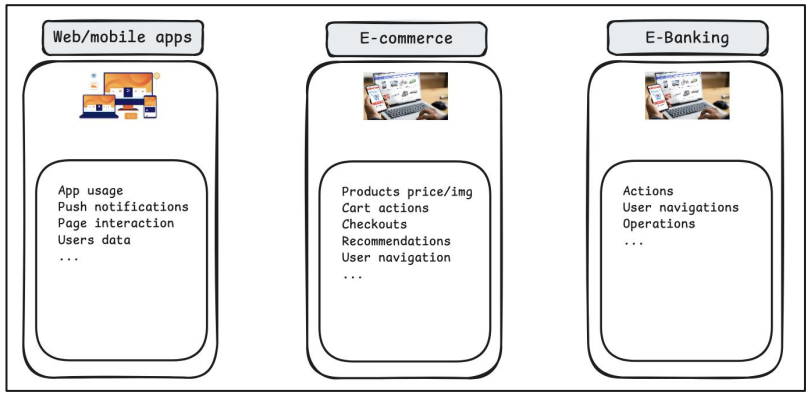
- Explosion of data from:
  - IoT



The evolution of data: from structured to big data

Big Data Era

- Explosion of data from:
  - Logs and clickstreams



Require new models for storage & processing : real time, various formats, ...

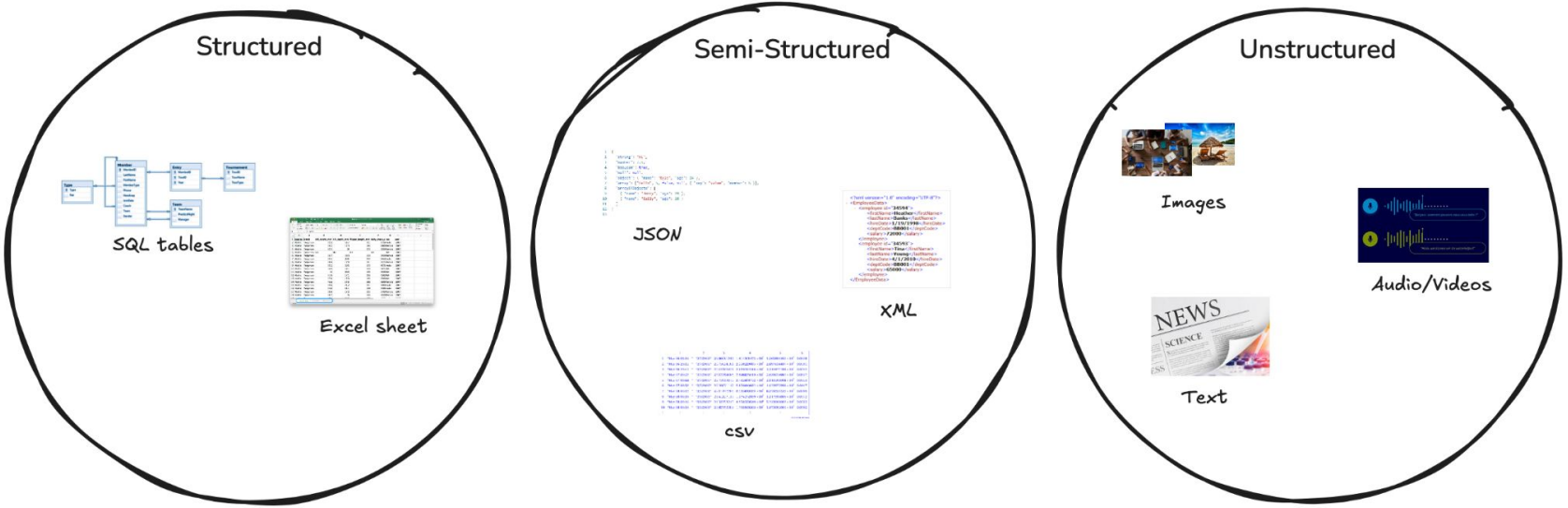
The evolution of data: from structured to big data

Characteristics of Big Data

- **Volume** – Massive data size (terabytes to petabytes)
- **Velocity** – Data generated in real-time
- **Variety** – Structured, semi-structured, unstructured
- **Veracity** – Quality & uncertainty of data
- **Value** – Turning data into insights

Transition to NoSQL

Types of Modern Data



Transition to NoSQL

Concept of NoSQL

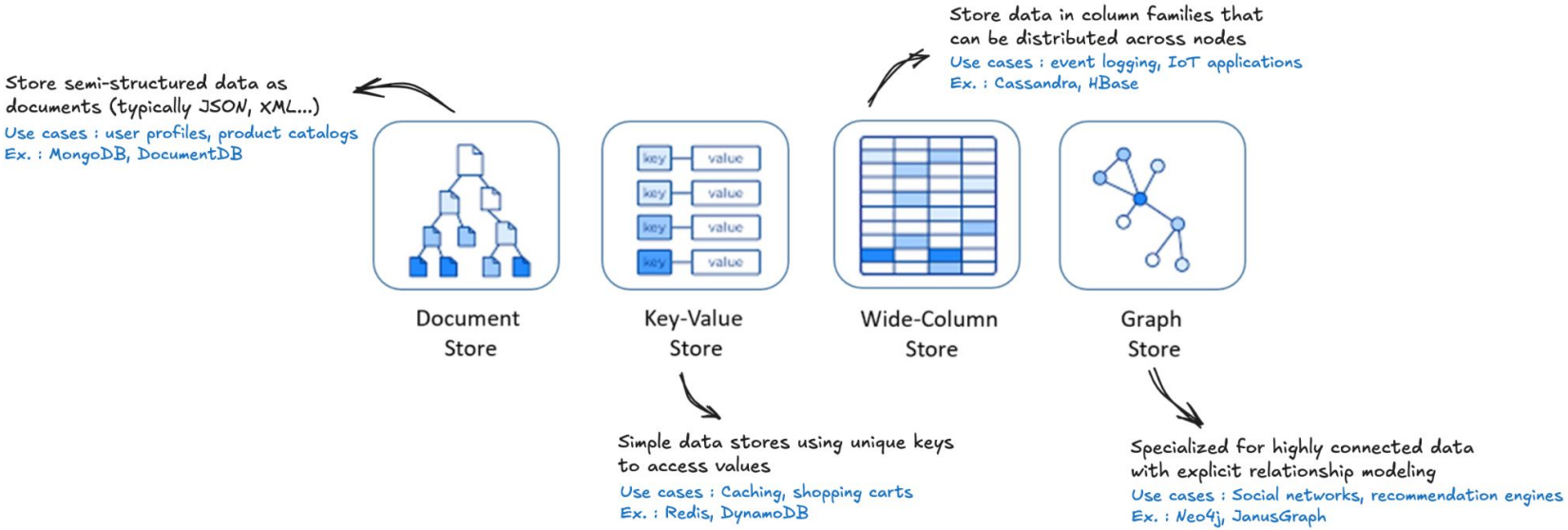
- **NoSQL** ⇔ **N**ot **O**nly **SQL**
- Databases that allows you to manage the limitations of RDBMS :
  - **Flexible schema** : can store different data types
  - **Horizontal scalability** : Easily scale out by adding more servers
  - **High performance for specific workloads** : massive data, real time
  - **Availability** and **partition tolerance** : distribute data over servers

Part 2 :

Introduction to NoSQL Databases

NoSQL Databases

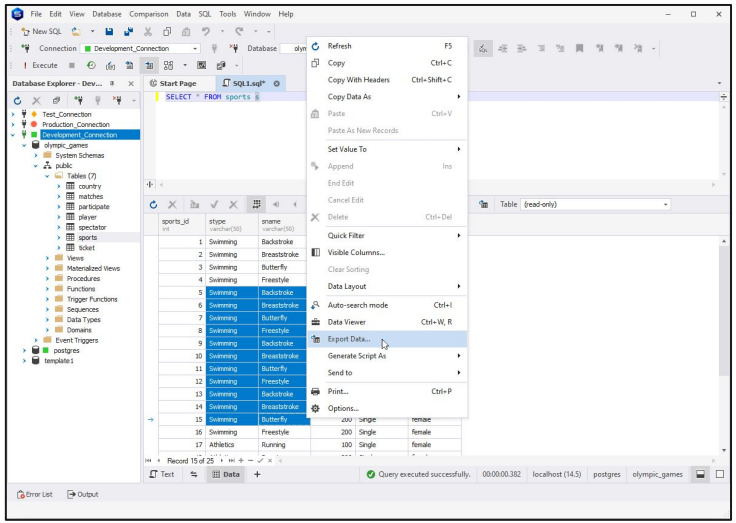
Classification



NoSQL Databases

Classification

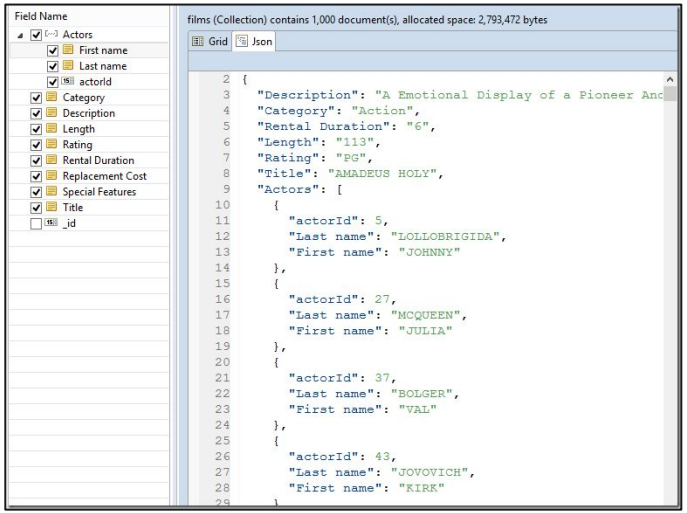
- Relational databases :



NoSQL Databases

Classification

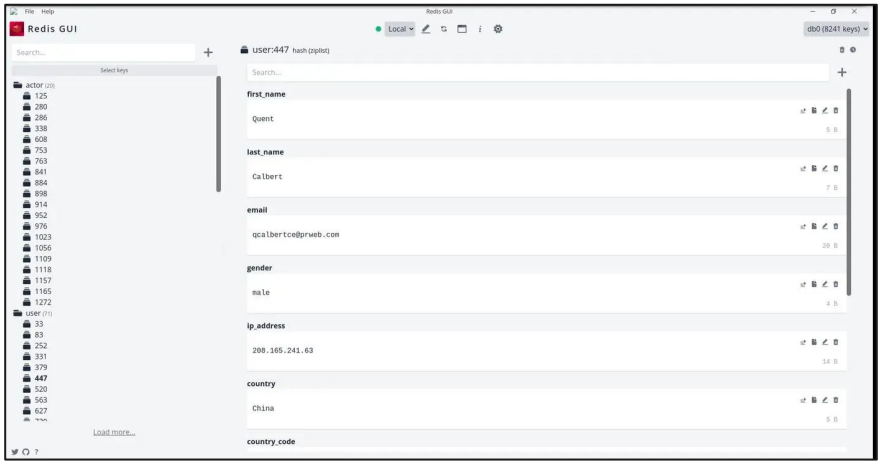
- Document databases :



NoSQL Databases

Classification

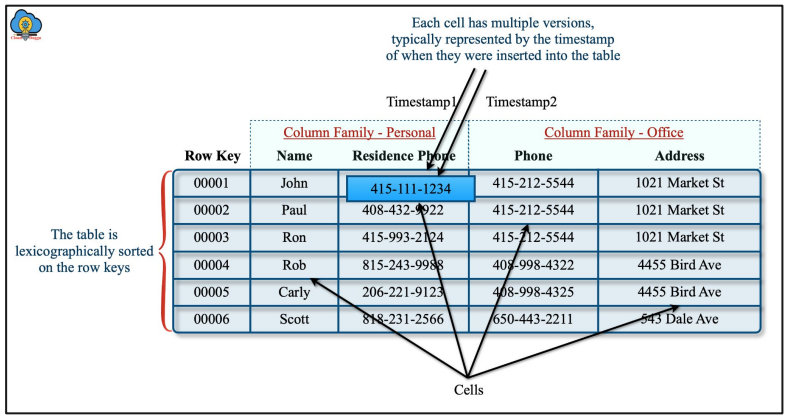
- Key-value databases :



NoSQL Databases

Classification

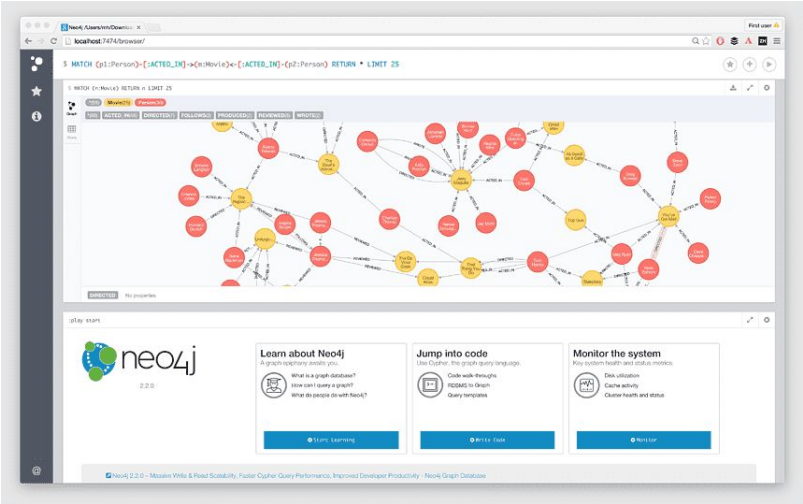
- Column-family databases :



NoSQL Databases

Classification

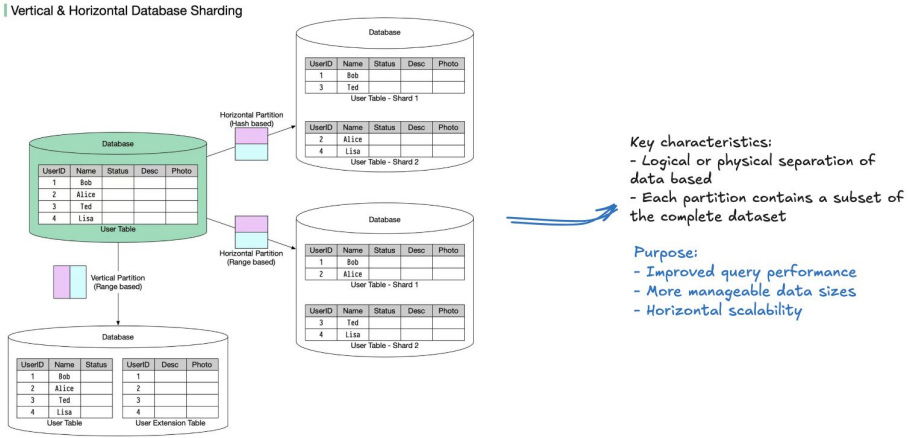
- Graph databases :



NoSQL Databases

Characteristics of databases

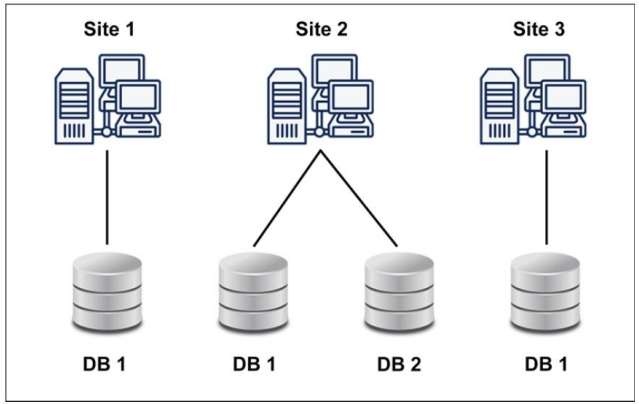
- **Data partitioning**:
  - Data divided into smaller, more manageable chunks (partitions/shards) based on a partitioning strategy.



NoSQL Databases

Characteristics of databases

- **Data distribution**
  - Data is spread across multiple physical machines or nodes, typically in different locations.



Key characteristics:
- Geographic separation between nodes
- Replication of data across locations for availability/redundancy

Purpose:
- High availability
- Disaster recovery
- Geographic performance optimization

NoSQL Databases

Characteristics of databases

- **The CAP Theorem** => in any distributed system, it is not possible to satisfy these 3 properties at the same time :
  - **Consistency** : All servers see the same data at the same time (if you write something, everyone instantly sees it)
  - **Availability** : Every request gets a response, even if it's not the latest data (the system is always up and replies, even during problems)
  - **Partition tolerance** : The system still works even if parts of it can't communicate (like when a network cable is cut)

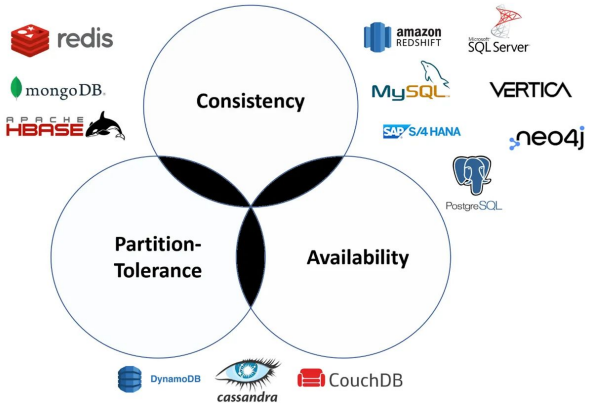
- **Rule** : We can only guarantee 2 out of the 3 at any moment, must sacrifice one if there's a failure (like a server crash or network problem)

NoSQL Databases

Characteristics of databases

- The CAP Theorem

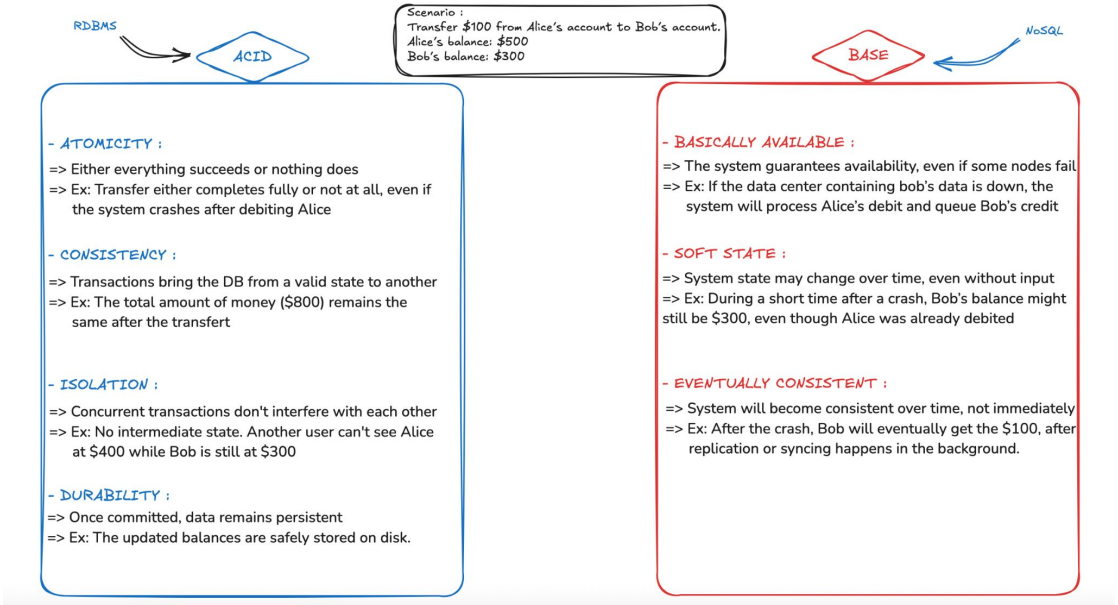
Choice	Example	Meaning
CP (Consistency + Partition)	HBASE, MongoDB	Always correct data, but might be unavailable during network issues
AP (Availability + Partition)	Amazon DynamoDB, Cassandra	Always responsive, but maybe outdated data sometimes
CA (Consistency + Availability)	Traditional databases (MySQL, PostgreSQL)	Needs perfect network – All clients can read and write on same data



NoSQL Databases

Characteristics of databases

- BASE Vs ACID



Part 3 :

Docker for Database Environments

Docker for Database Environments

Introduction to containerization

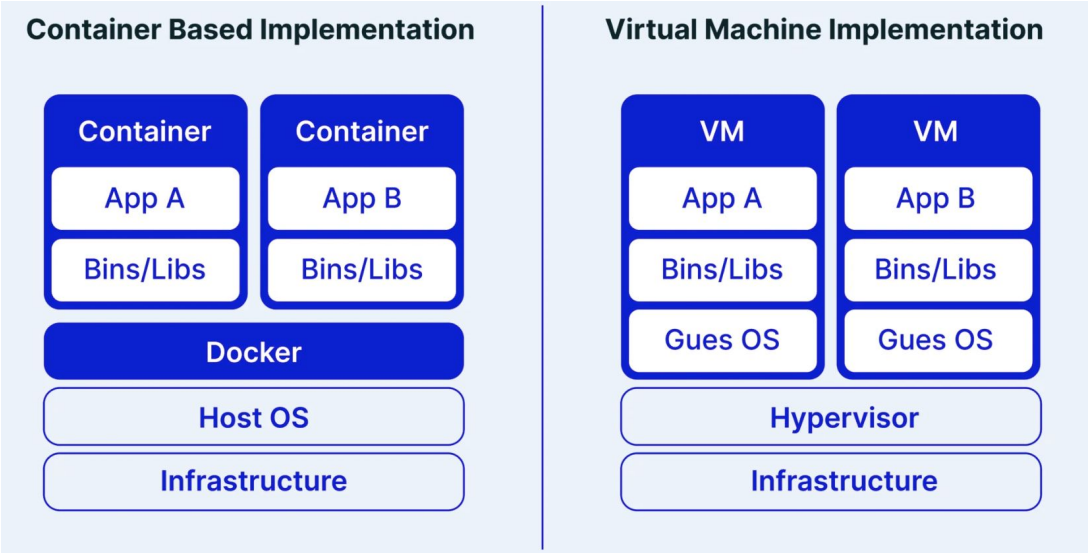
- **Containerization** is a method of packaging and running applications along with their dependencies in a lightweight, portable, and isolated environment called a container

- Different from **virtualization** which is a process of creating an abstraction layer over hardware, allowing a single computer to be divided into multiple virtual computers.

- **Container** : standalone virtual machine, but without the overhead of a full Operating System. It includes:
  - The application (e.g., a database)
  - Its dependencies (libraries, binaries, config files)
  - Everything needed to run, regardless of the host system

Docker for Database Environments

Introduction to containerization



Docker for Database Environments

Containerization for databases

Why containerization in the context of databases ?

- Easy & Consistent Deployment, Portability
  - Containers work across different environments: Linux, Mac, Windows, cloud platforms.
  - The database will behave the same way on your machine, in staging, in production…
- Fast Testing and Development
  - Developers can test multiple versions of a database without complex installs.
  - Infrastructure as code: define your full DB setup in code and redeploy it anywhere.
  - You can isolate test environments easily.

⚠️ **Considerations for Production :**

- Containers are **ephemeral**: by default, data is lost when the container is deleted.
- So for databases, it’s critical to use volumes to persist data

Docker for Database Environments

Docker solution

🐳 **What is Docker?**

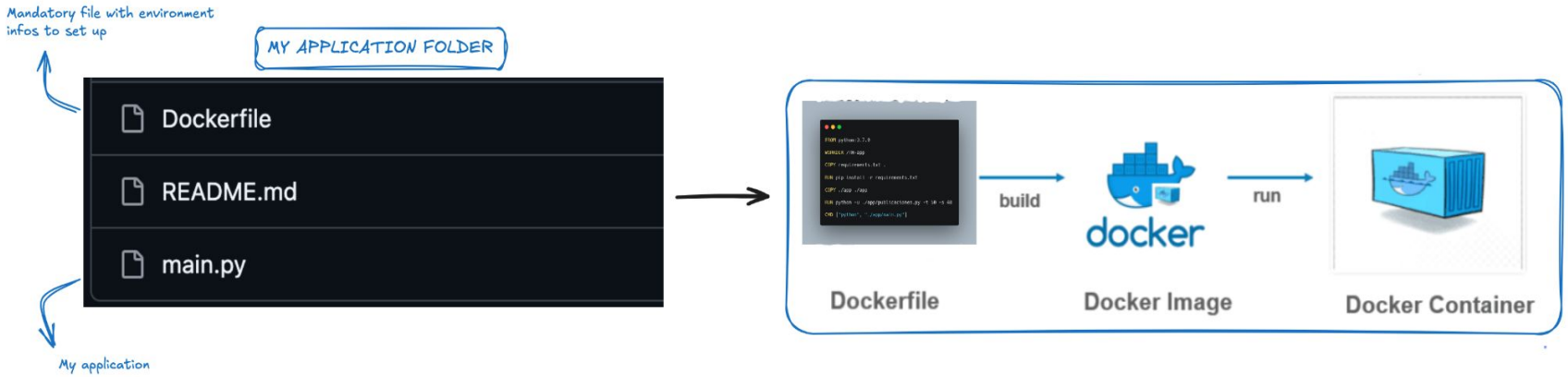
- An open-source platform that enables you to automate the deployment, scaling, and management of applications using lightweight & portable containers
- A Docker **container** bundles an application along with all its dependencies (libraries, configuration files, system tools) so it can run reliably across different computing environments

How Docker Works?

- A **Dockerfile** describes the environment and dependencies (possible to retrieves samples online)
- You build an image from that file
- You run that image as a container

Docker for Database Environments

Docker solution

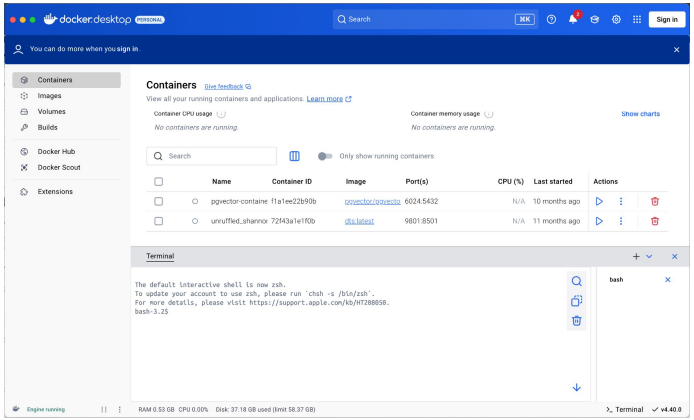


Docker for Database Environments

Installing docker

Desktop version :

- **Windows** : https://docs.docker.com/desktop/setup/install/windows-install/
- **Mac OS** : https://docs.docker.com/desktop/setup/install/mac-install/

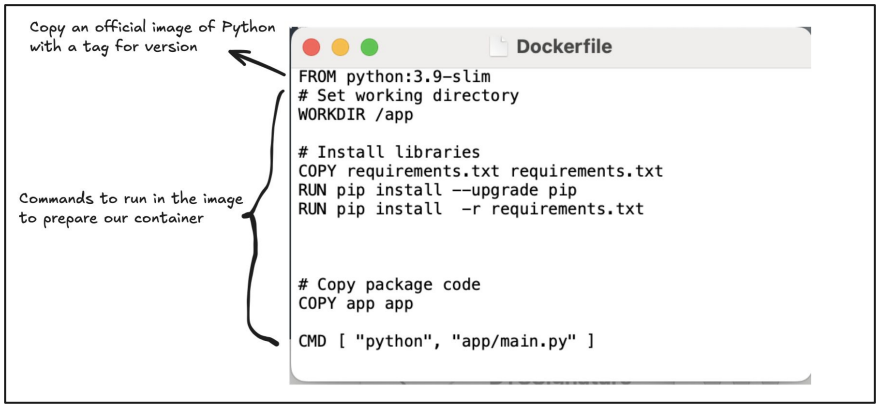


Docker for Database Environments

Building and running containers

Suppose we have a Python code (*main.py*) that we want to run inside a docker container.
We must complete **3 Steps** :

- **1-** Create a **Dockerfile** in the same directory

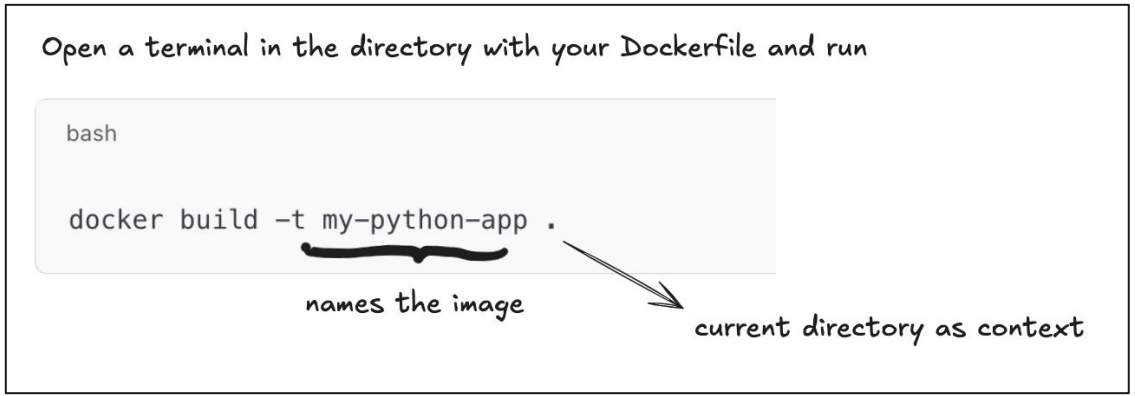


Docker for Database Environments

Building and running containers

Suppose we have a Python code (*main.py*) that we want to run inside a docker container.
We must complete **2 Steps** :

- **2- Build** the Docker image



Docker for Database Environments

Building and running containers

Suppose we have a Python code (*main.py*) that we want to run inside a docker container.
We must complete **3 Steps** :

- **3- Run** the Docker image in a container

NB : the container will stop running after completing if the app is only about running a script

Docker for Database Environments

Building and running containers

Other docker commands

- **List containers:**
  - `docker ps`
- **Check logs:**
  - `docker logs my-container`
- **Stop a container:**
  - `docker stop my-container`
- **Remove a container:**
  - `docker rm my-container`
- **Copy a remote image from Docker hub :**
  - `docker pull `THE_OFFICIAL_IMAGE_NAME``

Docker for Database Environments

Building and running containers

Exercise

- **Basic docker app :**
  - Build & run your own docker image that executes a python script. This project can inspire you : https://github.com/ababacaryoro/nosql_course/tree/dev/Docker-init

- **Docker app with Jupyter notebook:**
  - Create a docker app running a Jupyter notebook application on **port 9988** of your computer
  - Mount with your app a local folder containing your notebooks when running your application

Docker for Database Environments

Docker Compose : multi-container applications

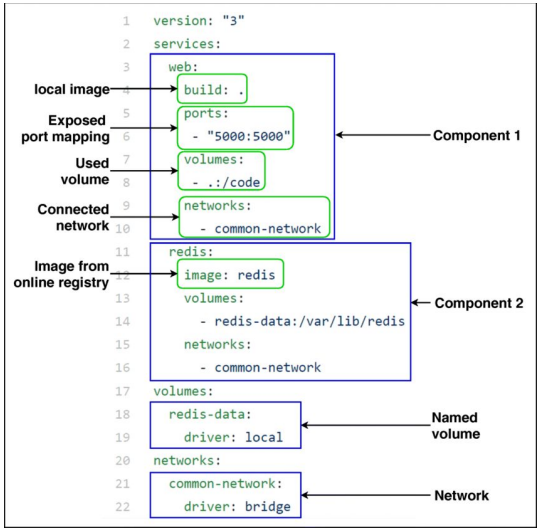
- When an application needs to run multiple services for different tasks => multiple docker applications
- Instead of running individual ***docker run*** commands for each service, Docker Compose lets you define them in a file called ***docker-compose.yml***
- Ex : a web app that needs a database, a cache, and maybe a background worker.
- Benefits :
  - Centralized configuration for all containers
  - Easy to bring up/down a multi-container setup
  - Networking between containers is automatic
  - Supports volume mounting, environment variables, port mappings

Docker for Database Environments

Docker Compose : multi-container applications

Steps to create a multi-container application

- **1-** Create the ***docker-compose.yml*** file

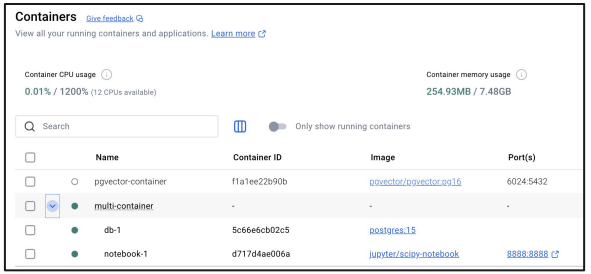


Docker for Database Environments

Docker Compose : multi-container applications

Steps to create a multi-container application

- **2-** Start the app
  - `docker-compose up` or `docker compose up` in MacOS



- Other commands :
  - Stop the App: `docker-compose down`
  - Build services from Dockerfiles : `docker-compose build`
  - View logs from services: `docker-compose logs`
  - List containers : `docker-compose ps`

Docker for Database Environments

Docker Compose : multi-container applications

Exercise :

- Create a multi-container application running with a Jupyter notebook and a PostgreSQL app, with the following specifications :
  - Mount/create a volume for the folder of your notebooks with the Jupyter app
  - Create password & username with the PostgreSQL app (use a .env file)
  - Mount a persistent volume for the database
  - Run the container & open a new notebook with the Jupyter app
  - Using the notebook :
    - Connect to PostgreSQL and create a database “school”
    - Create 2 tables : students (columns name, birth date, class) and course (columns name, duration, level)
    - Add random data to theses tables
    - Check if was correctly added (load tables)

Part 4 :

Key-Value Stores with Redis

Key-Value Stores with Redis

Key-value store concepts

- A **Key-Value Store** is a simple type of NoSQL database where each piece of data is stored as a pair:
  - **Key** → Value

- **Structure** :
  - **Key**: Unique identifier (e.g., a string or number)
  - **Value**: Any kind of data (e.g., string, JSON, binary, image)

- **Example** :

```
"user:1001" → { "name": "Alice", "age": 29 }
"session:xyz" → "abc123token"
```

Key-Value Stores with Redis

Key-value store concepts

Use cases

Key-Value Stores with Redis

Key-value store concepts

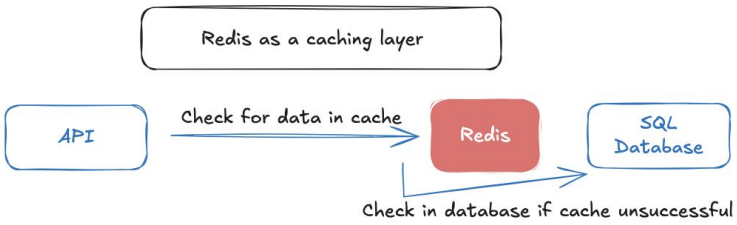
- **Characteristics** :
  - **Very fast** due to O(1) access
  - **Scalable** and simple to distribute across servers
  - **Easy to use** for developers

- **Limitations** :
  - No complex queries (e.g., JOINs)
  - Hard to model relationships
  - Not ideal for structured, relational data

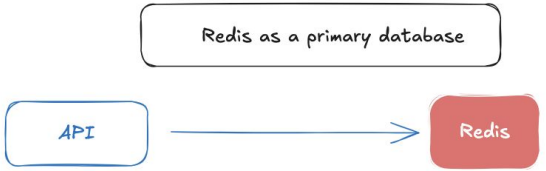
Key-Value Stores with Redis

Redis DB

- **Redis** (**Re**mote **Di**ctionary **S**erver) : an open-source database management system designed to store, retrieve, and manipulate various key/value pair data
- Can be configured to store data on **disk**, however, it is primarily used for **in-memory** data management, making it much faster
- **Usage** :



Data is stored in memory => in cash of crash, they will be lost in Redis but still available in the database

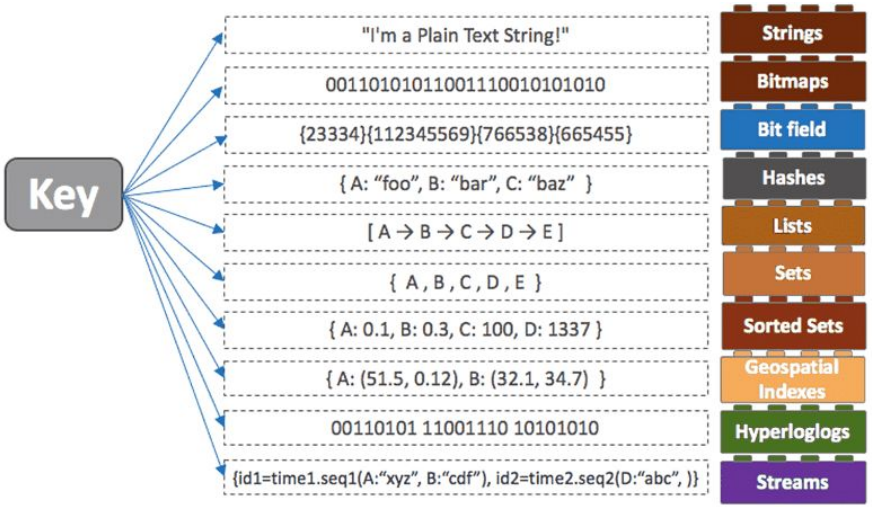


Data can be stored in disk => data is persisted and a system of replication is used to make data durable and available

Key-Value Stores with Redis

Redis DB

- **Redis data types :**



Key-Value Stores with Redis

Redis DB

- **Redis data organisation :**
  - No tables, collections or schema : just objects stores using keys/value system
- Simulate common **data organization** :
  - Use **data structures** and **keys naming conventions** to better manipulate data
- Common Redis **key naming conventions** :
  - Use namespacing with colon separators (:) -> ‘user:123’ ; ‘session:abc456’
  - Use lowercase letters and consistent casing : ‘product:456’ instead of ‘Product:456’
  - Prefix by entity type or application : ‘app1:user#1001’ ; ‘app2:user#1001’ (**colon** for namespaces, **sharp** for IDs)
  - Avoid spaces and special characters : Use underscores _ or colons : instead.
- **TTL** (Time To Live) : defines how long a key should exist before it automatically expires and is deleted from the Redis database.
  - Can be defined when creating the (key, value)

Key-Value Stores with Redis

Hands-on Redis

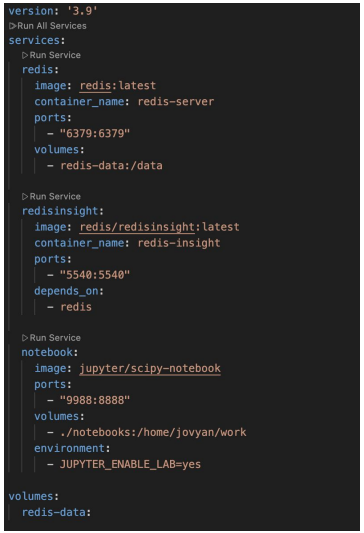
- **Installing :**
  - Desktop client : https://redis.io/open-source/
  - Redis Cloud : https://redis.io/cloud/
  - **Docker : https://redis.io/docs/latest/operate/oss_and_stack/install/install-stack/docker/**

- **TODO** : install using docker compose
  - Install redis image => “`image: redis:latest`“
  - Add redis insight (GUI client) => “`image: redis/redisinsight:latest`”
  - Add Jupyter notebook

Key-Value Stores with Redis

Hands-on Redis

- **TODO** : install using docker compose

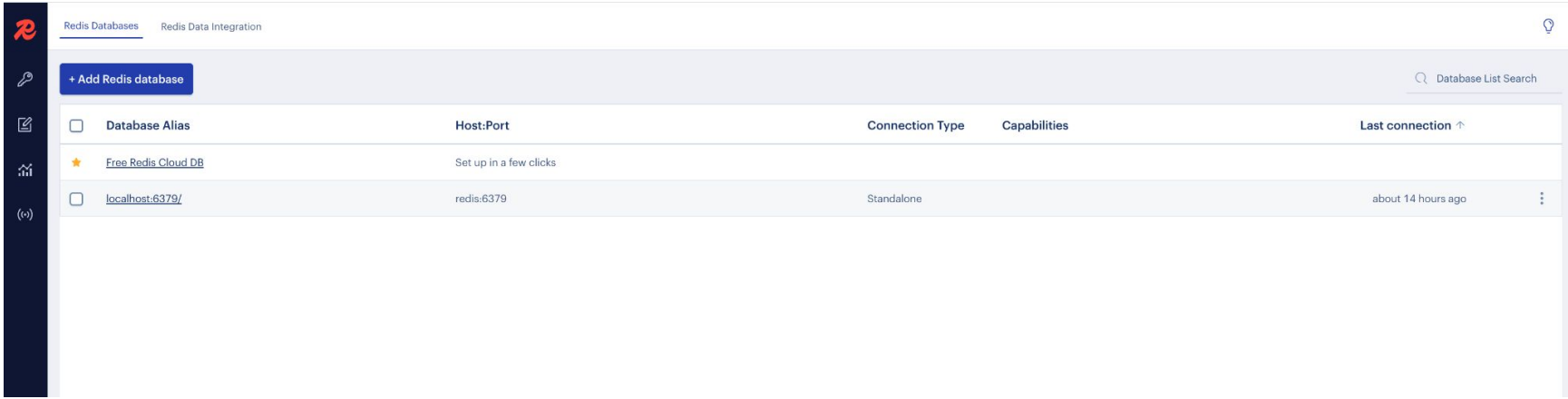


```yaml
version: '3.9'
▷Run All Services
services:
  ▷Run Service
  redis:
    image: redis:latest
    container_name: redis-server
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data

  ▷Run Service
  redisinsight:
    image: redis/redisinsight:latest
    container_name: redis-insight
    ports:
      - "5540:5540"
    depends_on:
      - redis

  ▷Run Service
  notebook:
    image: jupyter/scipy-notebook
    ports:
      - "9988:8888"
    volumes:
      - ./notebooks:/home/jovyan/work
    environment:
      - JUPYTER_ENABLE_LAB=yes

volumes:
  redis-data:
```

Key-Value Stores with Redis

Hands-on Redis

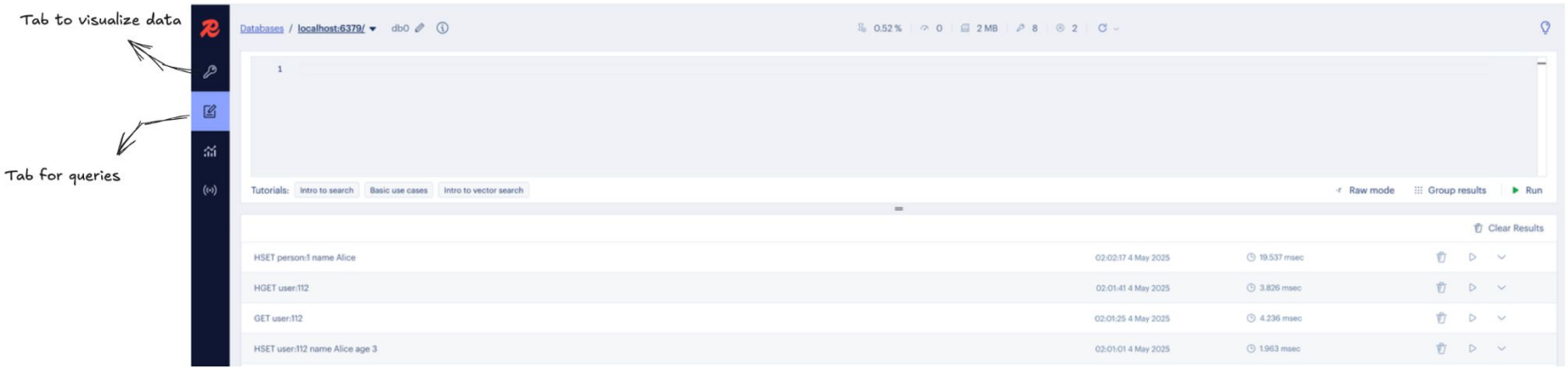
- **Redis insight** GUI client



Key-Value Stores with Redis

Hands-on Redis

- **Redis insight** GUI client



Key-Value Stores with Redis

Hands-on Redis

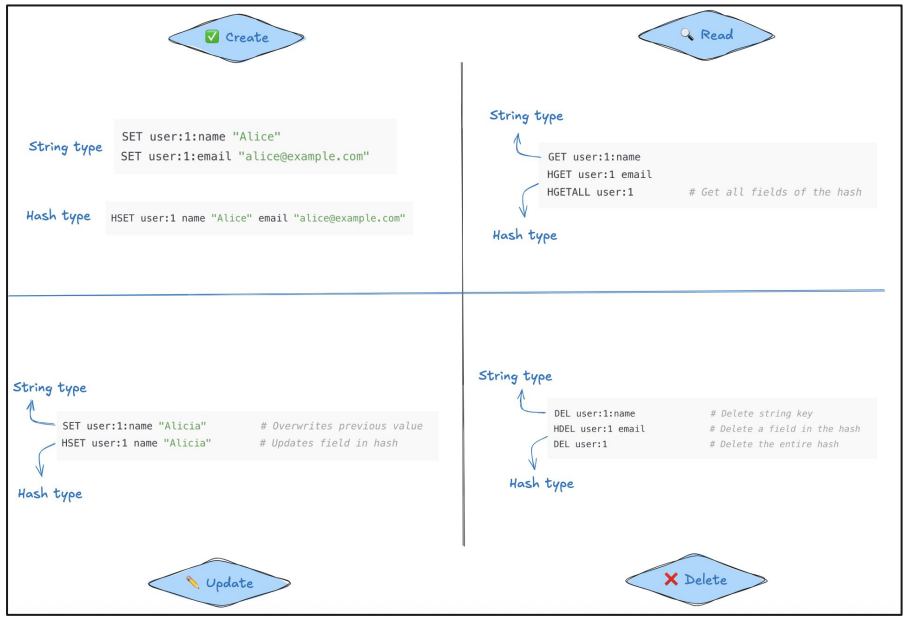
- **Basic commands :** Each data structure in Redis has its own unique set of operations that can be performed on it :

Category	Command	Description	Example
🔑 Keys	`SET key value`	Store a value	`SET user:1 "Alice"`
	`GET key`	Retrieve a value	`GET user:1`
	`DEL key`	Delete a key	`DEL user:1`
	`EXISTS key`	Check if key exists	`EXISTS user:1`
	`EXPIRE key seconds`	Set TTL	`EXPIRE user:1 60`
	`TTL key`	Get remaining TTL	`TTL user:1`
📚 Strings	`APPEND key value`	Add to string	`APPEND user:1 " Smith"`
🔢 Incr	`INCR key` / `DECR key`	Increment/Decrement	`INCR counter`
🗂️ Hashes	`HSET key field value`	Set hash field	`HSET user:1 name Alice`
	`HGET key field`	Get hash field	`HGET user:1 name`
📃 Lists	`LPUSH key value`	Add to list start	`LPUSH todos "buy milk"`
	`LRANGE key 0 -1`	Get all list items	`LRANGE todos 0 -1`
🔢 Sets	`SADD key value`	Add to set	`SADD tags redis`
	`SMEMBERS key`	Get set members	`SMEMBERS tags`

Key-Value Stores with Redis

Hands-on Redis

- **CRUD (Create, Read, Update, Delete):**



Key-Value Stores with Redis

Hands-on Redis

- **Retrieve multiple keys :**

Key-Value Stores with Redis

Hands-on Redis

- **Basic commands :** using Python client


```
!pip install redis
import redis
r = redis.Redis(host='redis', port=6379, db=0)
```

Create

```
# Set a key
r.set("user:1", "Alice")

# Set multiple keys
r.mset({"user:2": "Bob", "user:3": "Charlie"})
```

Read

```
# Get a value
r.get("user:1")  # b'Alice'

# Get multiple values
r.mget(["user:1", "user:2"])  # [b'Alice', b'Bob']

# List all matching keys
r.keys("user:*")  # [b'user:1', b'user:2', b'user:3']
```

Update

```
# Overwrite value
r.set("user:1", "Alice Smith")

# Increment value (e.g., score)
r.set("score", 100)
r.incr("score")     # 101
r.incrby("score", 5) # 106
```

Delete

```
# Delete a single key
r.delete("user:1")

# Delete multiple keys
r.delete("user:2", "user:3")
```

Key-Value Stores with Redis

Hands-on Redis

- **Application :**
  - Instructions : https://github.com/ababacaryoro/nosql_course/blob/dev/Redis/redis_practical_exercise.md
  - Actions to do :
    - Load data into Redis using Docker
    - Interact with Redis using Python
    - Write queries to analyze data

Part 5 :

Document Stores with MongoDB

Document Stores with MongoDB

Document Stores concepts

- A **Document Store** is a type of NoSQL database designed to store, retrieve, and manage semi-structured data in the form of documents, typically in JSON, BSON, or XML format.

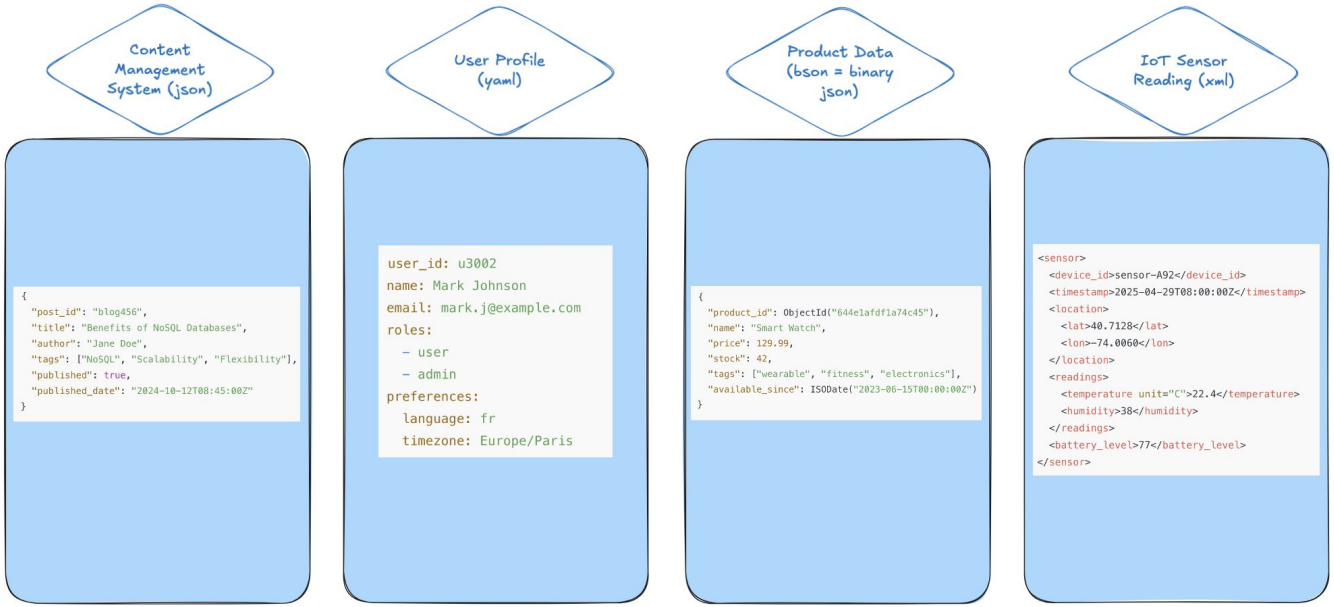
- Each **document** is :
  - A **self-contained unit** (like a row in SQL)
  - Can have **nested fields**, that can be **scalar** (string, number), **array**, or **nested objects**
  - Documents are grouped into **collections** (equivalent to tables in SQL)
- **Example of document** :

```
{
  "id": "user123",
  "name": "Alice",
  "email": "alice@example.com",
  "roles": ["admin", "editor"],
  "profile": {
    "age": 29,
    "location": "Paris"
  }
}
```

Document Stores with MongoDB

Document Stores concepts

Use cases



Document Stores with MongoDB

Document Stores concepts

- **Characteristics** :
  - **Schema-less:** documents can vary in structure.
  - **Rich query language:** supports filtering, projection, and aggregation
  - **Indexing:** you can index fields inside documents for fast queries
  - **Scalability:** designed for horizontal scaling (via sharding)

- **Limitations** :
  - **No strong schema enforcement** : documents can have inconsistent structures within 1 same collection
  - **Data duplication & redundancy** : data is often denormalized (copied across documents)
  - **Join limitations** : most document databases do not support joins natively like SQL

Document Stores with MongoDB

MongoDB

- **MongoDB** : a popular open-source, document-oriented NoSQL database

- **Key features** :
  - Stores data as **BSON** (binary JSON)
  - **Schema-less**: flexible document structure
  - Ideal for **semi-structured** or **evolving** data models

Document Stores with MongoDB

MongoDB

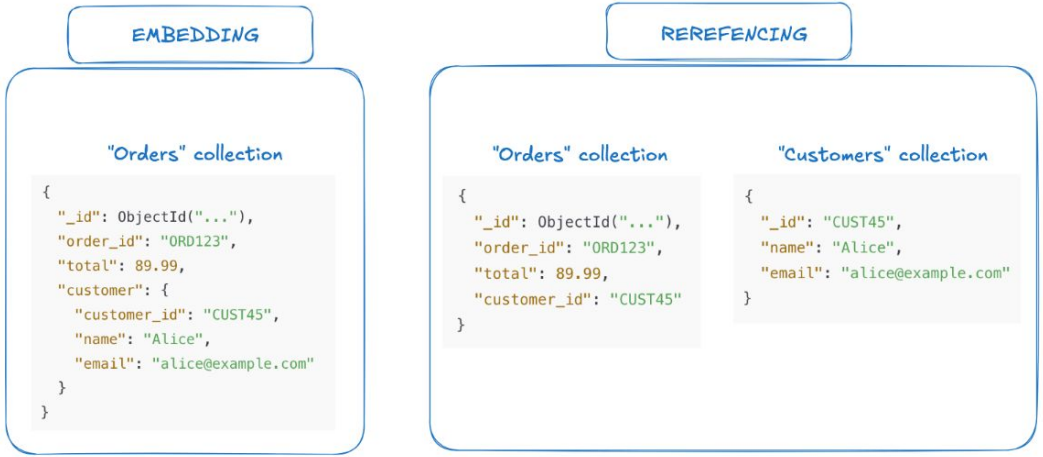
- **Data structure** :
  - **Database** → contains multiple **collections**
  - **Collection** → contains multiple **documents**
  - **Document** → a **JSON-like** object (`{ "name": "Alice", "age": 30 }`)
- **Common data types** :

Data Type	Example	Notes
String	`"name": "Alice"`	Most commonly used data type
Integer	`"age": 30`	32-bit or 64-bit depending on the server
Double	`"price": 19.99`	For floating-point numbers
Boolean	`"active": true`	`true` or `false`
Date	`"created_at": ISODate(...)`	Stores dates with millisecond precision
ObjectId	`"_id": ObjectId(...)`	Unique identifier for each document
Array	`"tags": ["tech", "news"]`	List of values
Embedded Document	`"address": { "city": "Paris" }`	Document inside a document
Null	`"deleted_at": null`	Explicitly represents a missing field

Document Stores with MongoDB

MongoDB

- **Data modeling options :**



🤔 **Decision Guide**

Criteria	Embed	Reference
Data is always read together	✅ Yes	❌ No (requires join or extra query)
Data changes frequently	❌ No (duplication risk)	✅ Yes (update once)
One-to-few relationship	✅ Embed is preferred	❌ Reference adds overhead
One-to-many or many-to-many	❌ Embedding can bloat document	✅ Use referencing

Document Stores with MongoDB

Hands-on MongoDB

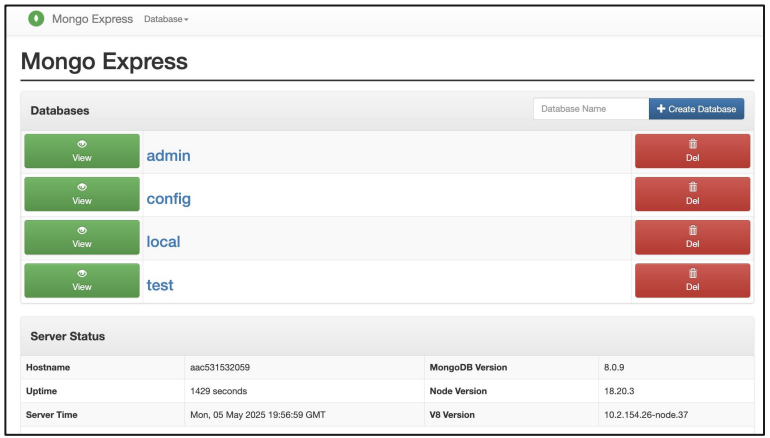
- **Installing :**
  - Desktop client : https://www.mongodb.com/docs/manual/installation/
  - **Docker :**
    - **https://hub.docker.com/_/mongo**
    - **https://www.mongodb.com/docs/manual/tutorial/install-mongodb-community-with-docker/**

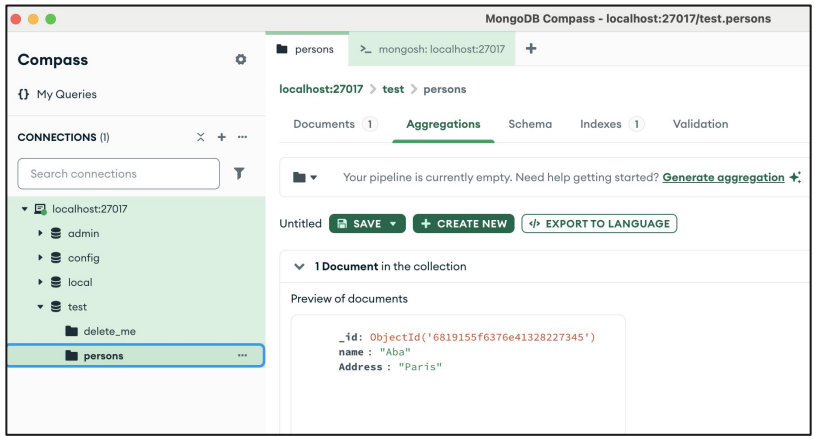
- **TODO** : install using docker compose
  - Install MongoDB image => "`image: mongo:latest`"
  - Add MongoBD (GUI client) => "`image: mongo-express:latest`" or install Mongo Compass (https://www.mongodb.com/try/download/compass)
  - Add Jupyter notebook

Document Stores with MongoDB

Hands-on MongoDB

- **MongoDB** GUI clients (MongoDB Express & MongoDB Compass)

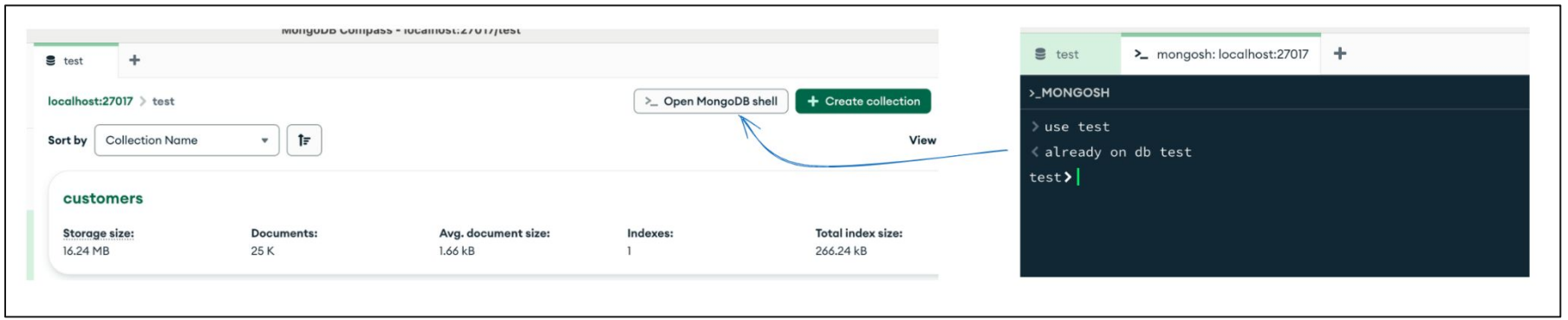




Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** MongoDB shell (***mongosh***) => on MongoDB Compass



Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** MongoDB shell (***mongosh***) => on MongoDB Compass

Database commands

```
// Show all databases
show dbs

// Switch to or create a new database
use myDatabase

// Show current database
db
```

Collection commands

```
// Show collections in the current database
show collections

// Create a new collection
db.createCollection("myCollection")

// Drop (delete) a collection
db.myCollection.drop()
```

Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** MongoDB shell (***mongosh***) : **CRUD**


```
// Insert one document
db.users.insertOne({ name: "Alice", age: 25 })

// Insert multiple documents
db.users.insertMany([
  { name: "Bob", age: 30 },
  { name: "Charlie", age: 35 }
])
```


```
// Find all documents
db.users.find()

// Find with filter
db.users.find({ age: { $gt: 30 } })

// Find one document
db.users.findOne({ name: "Alice" })
```

Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** MongoDB shell (***mongosh***) : **CRUD**

```
// Update one document
db.users.updateOne({ name: "Alice" }, { $set: { age: 26 } })

// Update multiple documents
db.users.updateMany({}, { $inc: { age: 1 } })  // Increment age for all users
```


```
// Delete one document
db.users.deleteOne({ name: "Bob" })

// Delete multiple documents
db.users.deleteMany({ age: { $lt: 30 } })
```

Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** MongoDB shell (***mongosh***) : **Filters**
  - Can be used inside the **.find()** method

Operator	Description	Example
`$eq`	Equal to	`{ age: { $eq: 25 } }`
`$ne`	Not equal to	`{ age: { $ne: 25 } }`
`$gt`	Greater than	`{ age: { $gt: 18 } }`
`$gte`	Greater than or equal to	`{ age: { $gte: 18 } }`
`$lt`	Less than	`{ age: { $lt: 30 } }`
`$lte`	Less than or equal to	`{ age: { $lte: 30 } }`
`$in`	In array of values	`{ age: { $in: [18, 25, 30] } }`
`$nin`	Not in array of values	`{ age: { $nin: [18, 25, 30] } }`

Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** MongoDB shell (***mongosh***) : **Filters**
  - Can be used inside the **.find()** method

Operator	Description	Example
`$eq`	Equal to	`{ age: { $eq: 25 } }`
`$ne`	Not equal to	`{ age: { $ne: 25 } }`
`$gt`	Greater than	`{ age: { $gt: 18 } }`
`$gte`	Greater than or equal to	`{ age: { $gte: 18 } }`
`$lt`	Less than	`{ age: { $lt: 30 } }`
`$lte`	Less than or equal to	`{ age: { $lte: 30 } }`
`$in`	In array of values	`{ age: { $in: [18, 25, 30] } }`
`$nin`	Not in array of values	`{ age: { $nin: [18, 25, 30] } }`

Document Stores with MongoDB

Hands-on MongoDB

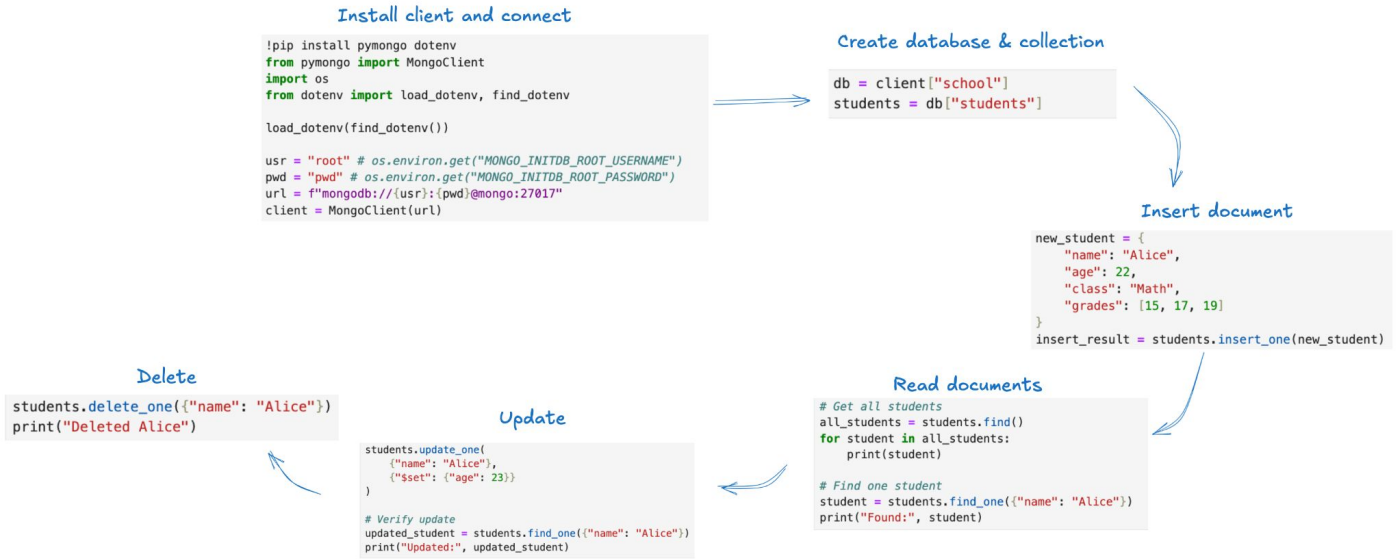
- **Basic commands :** MongoDB shell (***mongosh***) : **Filters**
  - Can be used inside the **.find()** method

Operator	Description	Example
`$and`	Logical AND	`{ $and: [{age: {$gt:18}}, {age: {$lt:30}}] }`
`$or`	Logical OR	`{ $or: [{age: {$lt:18}}, {age: {$gt:65}}] }`
`$not`	Negate a condition	`{ age: { $not: { $gt: 18 } } }`
`$nor`	None of the conditions are true	`{ $nor: [{age: {$gt:18}}, {name: "Alice"}] }`

Document Stores with MongoDB

Hands-on MongoDB

- **Basic commands :** using Python client



Document Stores with MongoDB

Hands-on MongoDB

- **Application 1 : MongoDB install & CRUD operations**
  - Instructions : https://github.com/ababacaryoro/nosql_course/blob/dev/MongoDB/mongodb_practical_exercise1.md
  - Actions to do :
    - Extract data, transform and load on MongoDB using Docker
    - Interact with data using Mongo shell
    - Package & Deploy your code

Document Stores with MongoDB

Hands-on MongoDB

- **Application 2 : minimal ETL**
  - Instructions : https://github.com/ababacaryoro/nosql_course/blob/dev/MongoDB/mongodb_practical_exercise2.md
  - Actions to do :
    - Extract data, transform and load on MongoDB using Docker
    - Interact with data using Mongo shell
    - Package & Deploy your code

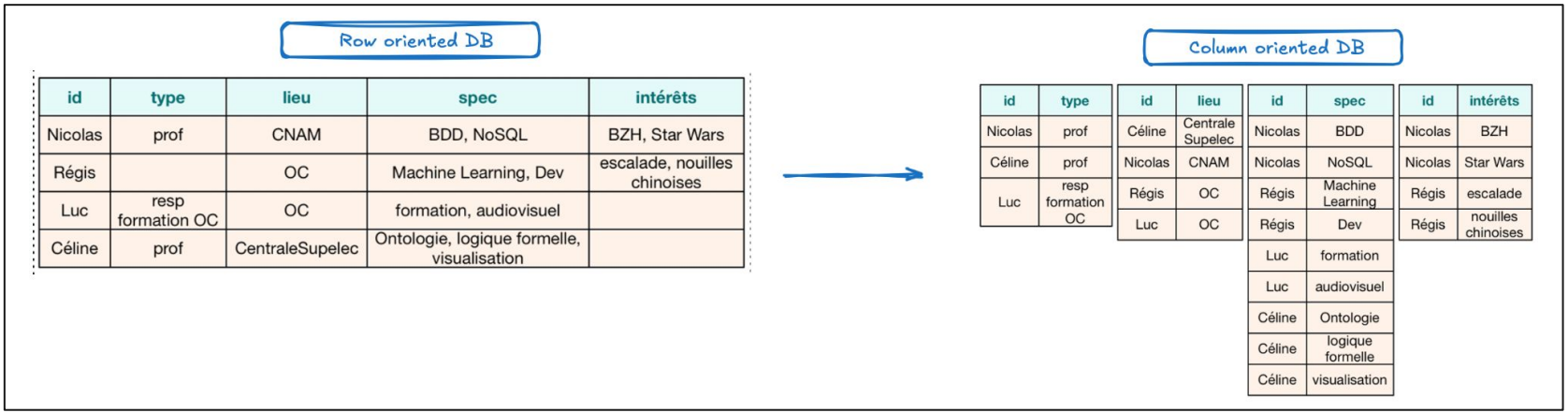
Part 6 :

Column-family Stores with HBASE

Column-family Stores

Column-family Stores concepts

- A **Column-family Store** is a type of NoSQL database that stores data in columns instead of rows.



id	type	lieu	spec	intérêts
Nicolas	prof	CNAM	BDD, NoSQL	BZH, Star Wars
Régis		OC	Machine Learning, Dev	escalade, nouilles chinoises
Luc	resp formation OC	OC	formation, audiovisuel	
Céline	prof	CentraleSupelec	Ontologie, logique formelle, visualisation	

id	type
Nicolas	prof
Céline	prof
Luc	resp formation OC

id	lieu
Céline	Centrale Supelec
Nicolas	CNAM
Régis	OC
Luc	OC

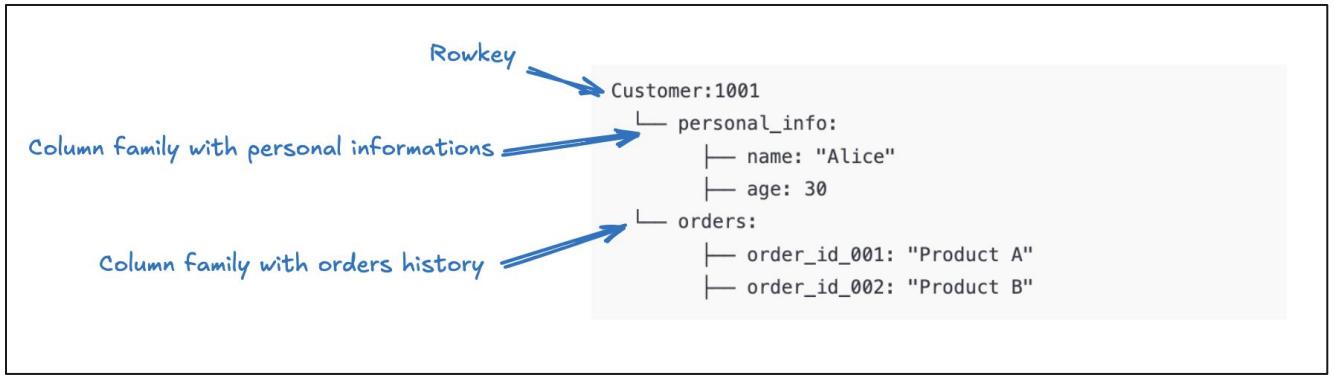
id	spec
Nicolas	BDD
Nicolas	NoSQL
Régis	Machine Learning
Régis	Dev
Luc	formation
Luc	audiovisuel
Céline	Ontologie
Céline	logique formelle
Céline	visualisation

id	intérêts
Nicolas	BZH
Nicolas	Star Wars
Régis	escalade
Régis	nouilles chinoises

Column-family Stores

Column-family Stores concepts

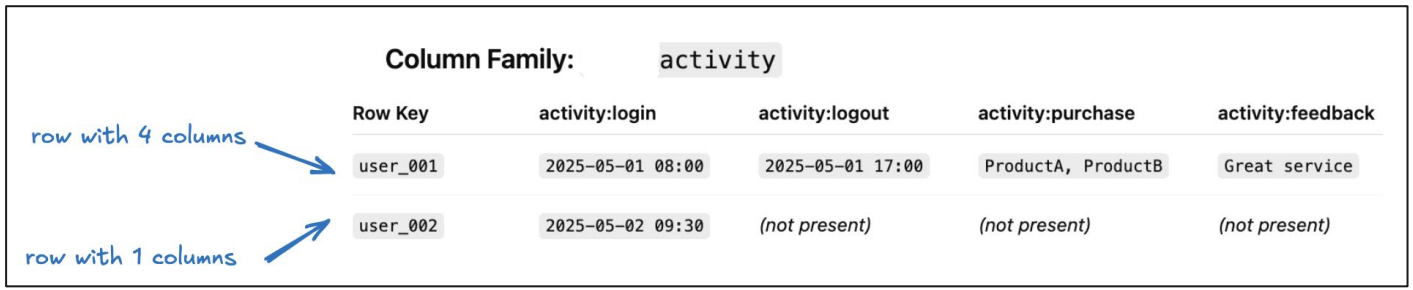
- **Row key :**
  - Each row is identified by a unique row key (like primary keys in Relational DB)
  - All the column data under this key is grouped together



Column-family Stores

Column-family Stores concepts

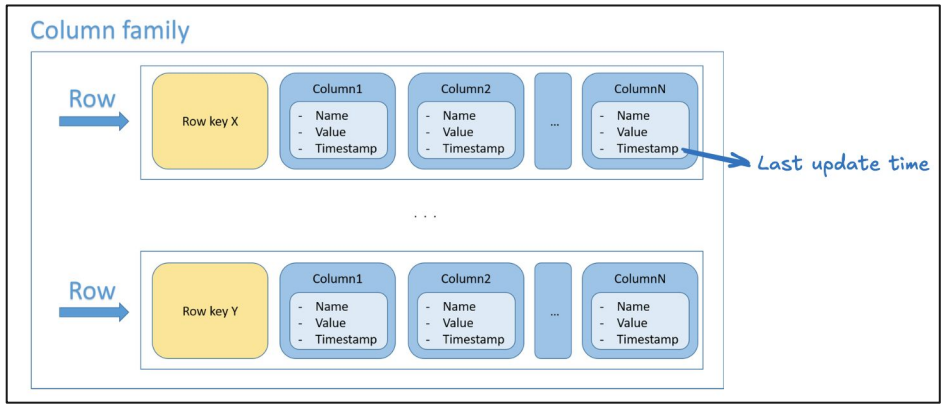
- **Column-family** :
  - Like a table in a relational database
  - It groups together a set of columns
  - Each row in a column family can have a different number of columns



Column-family Stores

Column-family Stores concepts

- **Column-family** :
  - New columns can be added if necessary : will be applied only on concerned rows
  - Every column has 3 characteristics : **name**, **value**, **timestamp**



Column-family Stores

Column-family Stores concepts

- **Characteristics** :
  - **Sparse Structure/flexibility :**
    - Not all rows need to have the same columns
    - Only non-empty cells are stored—saving space for sparse data
  - **Fast read/write:** especially when accessing specific columns only
  - **Scales horizontally:** easy to distribute across machines
  - **Scalability:** designed for horizontal scaling (via sharding)

- **Limitations** :
  - **No joins support** : e.g., SQL-like joins, subqueries, or aggregates
  - **Complex Data Modeling** : Mistakes in modeling can lead to high read/write amplification
  - **No multirow transactions** : atomic read/write

Column-family Stores

Hbase

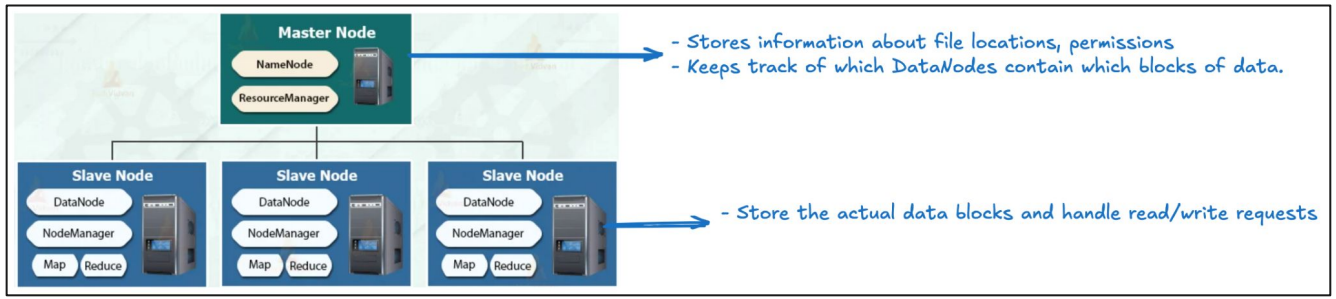
- **Hbase** : document-oriented NoSQL database for the Hadoop ecosystem
  - Part of the Apache Hadoop ecosystem, it integrates well with **MapReduce**, **Hive**, and **Spark**.
  - Runs on top of Hadoop Distributed File System (**HDFS**)

Feature	Description
Column-Oriented Store	Stores data in column families instead of rows (like in RDBMS).
Horizontally Scalable	Easily scales across commodity hardware clusters.
Schema-less	Flexible schema: each row can have different columns.
Real-Time Access	Supports fast reads and writes on very large datasets.

Column-family Stores

Hbase

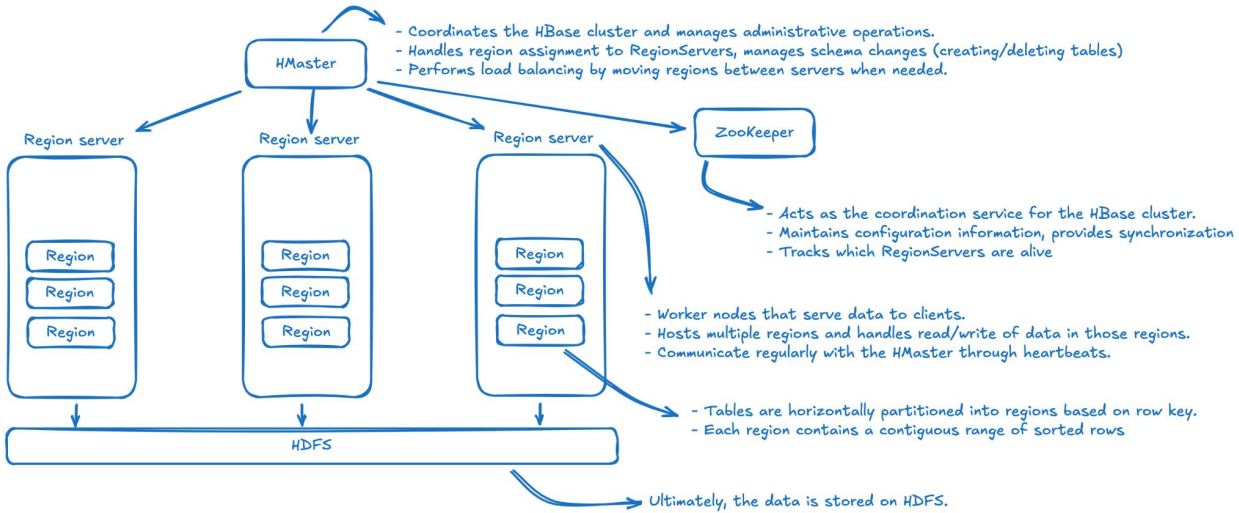
- **Hadoop ecosystem :**
  - **HDFS :**
    - A distributed storage system designed to store and manage large datasets across clusters of commodity hardware
    - Follows a **master-slave** architecture with two main types of nodes : NameNode (master) & DataNodes (slaves)



Column-family Stores

Hbase

- **Architecture :**



Column-family Stores

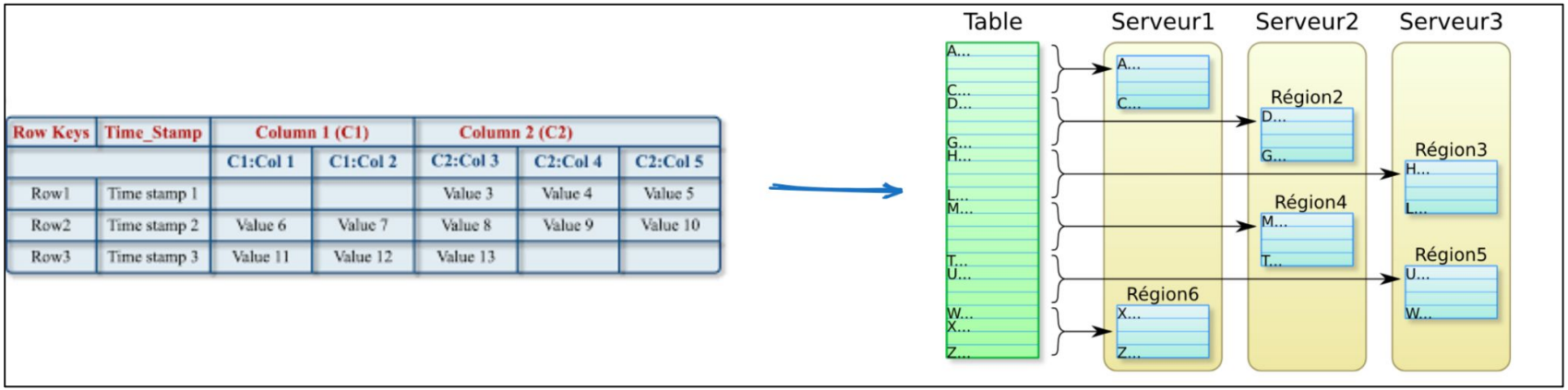
Hbase

- **Data storage structure**
  - **Tables** : HBase organizes data into tables, which are collections of rows identified by unique row keys. Tables are automatically partitioned into regions based on row key ranges.
  - **Column Families** : Columns are grouped into column families, which must be defined when creating a table. All columns in a family are stored together physically, enabling efficient access patterns.
  - **Columns (Column Qualifiers)**: Within each column family, you can have an unlimited number of columns, They don't need to be predefined - they can be added dynamically when data is inserted. Column names are specified as "family:qualifier" (for example, "personal:name" or "contact:email")
  - **Cells** : The intersection of a row and column creates a cell, which can store multiple versions of data with timestamps. Each cell contains the actual data value along with its timestamp for versioning.
  - **HFiles** : Data is ultimately stored in HFiles on HDFS. These are immutable files that contain sorted key-value pairs. Each column family's data is stored in separate HFiles, allowing for efficient column-family-level operations.

Column-family Stores

Hbase

- **Data storage structure**



Column-family Stores

Hbase

- **Data types :**
  - HBase is **schemaless** in terms of data types — everything is stored as a **byte** array
  - No **types** like integer, float, string, boolean, etc. — it just stores and retrieves bytes
  - You must handle **serialization** and **deserialization** yourself :
    - Convert it to a byte array before storing
    - Convert it back to an integer after retrieving
  - **Common best practices** :
    - Agree with your application logic on how to interpret data (e.g., rowkey is string, column value is a serialized JSON, int, float, etc.)
    - Or store all data in string format, then parse it back when treating it

Column-family Stores

Hbase

- **Data modeling tips**
  - **Row key design** → The most critical aspect of HBase data modeling since it determines data distribution and query performance
    - Use a unique and meaningful key (e.g., *userID#timestamp* or *deviceID#YYYYMMDD*)
    - Avoid hot-spotting by:
      - Salting (prefixing with a hash or region key) for region balance
      - Reversing IDs (e.g., for timestamps or incrementing keys)
      - Keep row keys reasonably short since they're stored with every cell. Long keys waste storage space and impact performance.

Column-family Stores

Hbase

- **Data modeling tips**
  - **Column family design**
    - **Minimize column families**: Use as few column families as possible (rule of thumb : 1-3 per table). Each column family creates separate storage files, and having too many can impact performance during operations like region splits and compactions.
    - **Group by access patterns**: Place columns that are frequently accessed together in the same column family. Separate columns that have different access patterns, storage requirements, or lifecycle management needs.

Column-family Stores

Hbase

- **Data modeling tips**
  - **Versionning & separating workspaces**
    - **Version management:** Decide how many versions of each cell to keep based on your application needs. More versions consume more storage but provide historical data access.
    - **Time-To-Live (TTL):** Set appropriate TTL values for different column families to automatically expire old data. This is crucial for managing storage costs in time-series applications.
    - **Namespaces** : Organize tables logically, especially in multi-tenant systems, using namespaces (like schemas in RDBMS)

Column-family Stores

Hands-on Hbase

- **Installing with docker :**

With Docker CLI

```
docker run -d \
  --name hbase \
  -p 16010:16010 \
  -p 2181:2181 \
  -p 9090:9090 \
  harisekhon/hbase
```

Using docker-compose

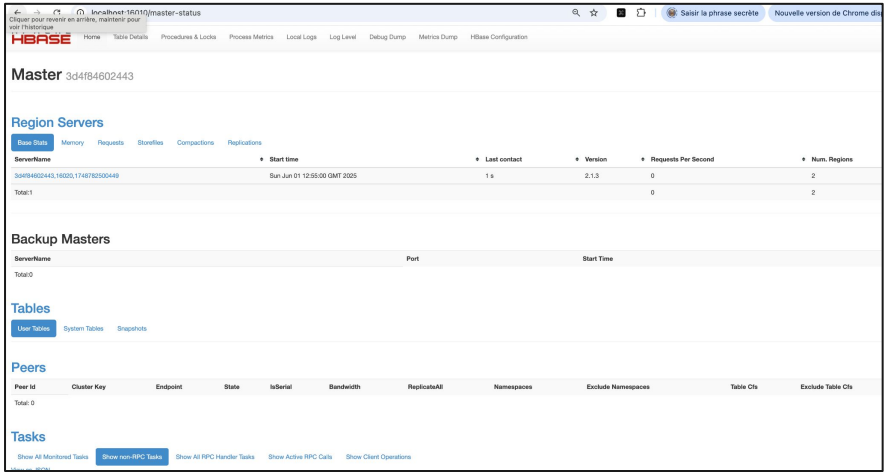
```
version: '3'

services:
  hbase:
    image: harisekhon/hbase
    container_name: hbase
    ports:
      - "16010:16010"  # HBase Web UI
      - "2181:2181"    # Zookeeper
      - "9090:9090"    # Thrift server
```

TODO : install it on your computer

Column-family Stores

Hands-on Hbase

- **Working with Hbase : differents options**
  - A **dynamic web page** that displays the status of the service and allows you to view the tables

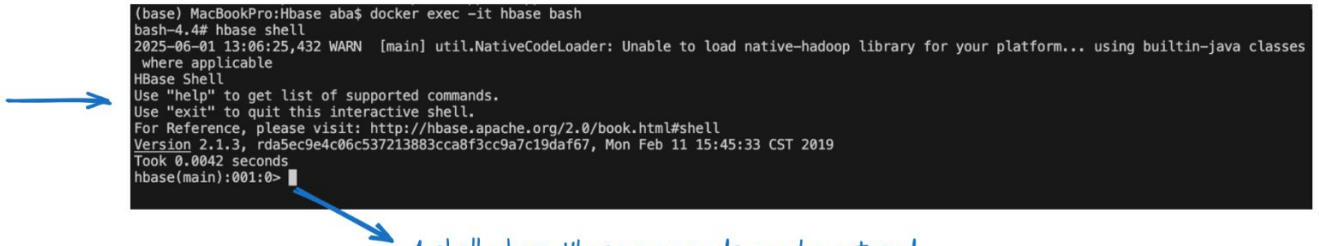


Column-family Stores

Hands-on Hbase

- **Working with Hbase : differents options**
  - A **shell** where commands can be written to interact with the database

```
docker exec -it hbase bash
hbase shell
```



Column-family Stores

Hands-on Hbase

- **Working with Hbase : differents options**
  - An **API** to interface with programming languages like Python

```
!pip install happybase
```

```
import happybase

conn = happybase.Connection('localhost', port=9090)
conn.open()

conn.create_table('test', {'cf1': dict()})
table = conn.table('test')
table.put('row1', {b'cf1:name': b'Alice'})
print(table.row('row1'))
```

Port of thrift server defined in the docker compose

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - Setup & Namespace management

```bash
# List namespaces
list_namespace

# Create namespace
create_namespace 'my_namespace'

# List tables in namespace
list_namespace_tables 'my_namespace'

# Create table in namespace
create 'my_namespace:table_name', 'cf1'
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - Tables management

```bash
# Create a table
create 'my_namespace:my_table', 'column_family1', 'column_family2'

# List all tables
list

# Describe a table
describe 'my_namespace:my_table'

# Disable a table (required before deletion)
disable 'my_namespace:my_table'

# Drop a table
drop 'my_namespace:my_table'
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - CRUD operations

```
# Insert data (put)
put 'my_namespace:my_table', 'row1', 'column_family1:column1', 'value1'

# Read data (get)
get 'my_namespace:my_table', 'row1'

# Scan table
scan 'my_namespace:my_table'

# Delete a cell
delete 'my_namespace:my_table', 'row1', 'column_family1:column1'

# Delete an entire row
deleteall 'my_namespace:my_table', 'row1'
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - **C**reate/**U**pdate operations

```bash
# Basic put operation
put 'table_name', 'row_key', 'column_family:column', 'value'

# Example with real data
put 'users', 'user001', 'personal:name', 'John Doe'
put 'users', 'user001', 'personal:age', '30'
put 'users', 'user001', 'contact:email', 'john@example.com'

# Put with timestamp
put 'users', 'user001', 'personal:name', 'John Smith', 1234567890
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - **R**ead operations (one row)

```bash
# Get entire row
get 'table_name', 'row_key'

# Get specific column
get 'users', 'user001', 'personal:name'

# Get column family
get 'users', 'user001', 'personal'

# Get with timestamp
get 'users', 'user001', {TIMESTAMP => 1234567890}

# Get specific versions
get 'users', 'user001', {COLUMN => 'personal:name', VERSIONS => 3}
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - **R**ead operations (multiple rows)

```bash
# Scan entire table
scan 'table_name'

# Scan with row key range
scan 'users', {STARTROW => 'user001', STOPROW => 'user999'}

# Scan specific column family
scan 'users', {COLUMNS => 'personal'}

# Scan with filter
scan 'users', {FILTER => "ValueFilter(=, 'binary:John')"}

# Limit results
scan 'users', {LIMIT => 10}

# Scan with time range
scan 'users', {TIMERANGE => [1234567890, 1234567999]}
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - **R**ead operations (filtering rows) : Doc → https://docs.cloudera.com/documentation/enterprise/5-6-x/topics/admin_hbase_filtering.html

```
# Value filter
scan 'users', {FILTER => "ValueFilter(=, 'binary:30')"}

# Row key filter
scan 'users', {FILTER => "RowFilter(=, 'regexstring:user00[1-5]')"}

# Column prefix filter
scan 'users', {FILTER => "ColumnPrefixFilter('personal')"}

# Multiple filters
scan 'users', {FILTER => "RowFilter(=, 'regexstring:user.*') AND ValueFilter(=, 'binary:John')"}

# Single column value filter
scan 'users', {FILTER => "SingleColumnValueFilter('personal', 'age', >=, 'binary:25')"}
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - **D**elete operations

```bash
# Delete specific cell
delete 'table_name', 'row_key', 'column_family:column'

# Delete column family
delete 'users', 'user001', 'personal'

# Delete entire row
deleteall 'table_name', 'row_key'

# Delete with timestamp
delete 'users', 'user001', 'personal:name', 1234567890
```

Column-family Stores

Hands-on Hbase

- **Basic commands :** Hbase shell
  - Table status & description

```bash
# Check if table is enabled
is_enabled 'my_namespace:my_table'

# Check if table exists
exists 'my_namespace:my_table'

# Count number of rows
count 'my_namespace:my_table'

# Truncate (delete all data)
truncate 'my_namespace:my_table'
```

Column-family Stores

Hands-on Hbase

- **Interacting with HBase in Python : CRUD operations**

Connect to Hbase

```
import happybase

connection = happybase.Connection('localhost', port=9090)
connection.open()
```

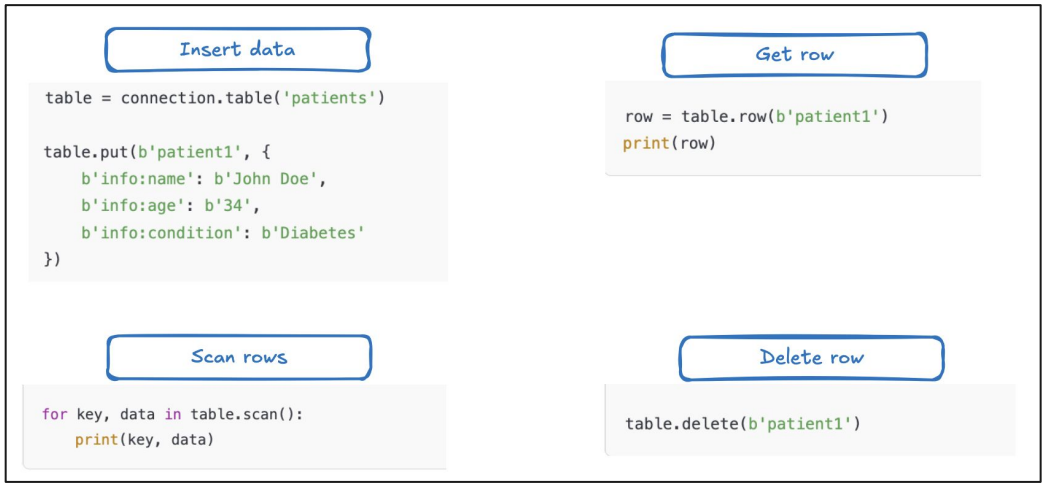
Create a table

```
connection.create_table(
    'patients',
    {'info': dict()}  # Column family 'info'
)
```

Column-family Stores

Hands-on Hbase

- **Interacting with HBase in Python : CRUD operations**



Column-family Stores

Hands-on Hbase

- **Interacting with HBase in Python : CRUD operations**
  - Insert multiple data in bulk

```
with table.batch() as b:
    b.put(b'patient2', {b'info:name': b'Alice'})
    b.put(b'patient3', {b'info:name': b'Bob'})
```

Column-family Stores

Hands-on Hbase

- **Application 1 : Hbase install & CRUD operations**
  - Instructions : https://github.com/ababacaryoro/nosql_course/blob/dev/Hbase/hbase_practical_exercise1.md
  - Actions to do :
    - Extract data, transform and load on MongoDB using Docker
    - Interact with data using Mongo shell
    - Package & Deploy your code

Column-family Stores

Hands-on Hbase

- **Application 2 : minimal ETL**
    - Instructions : https://github.com/ababacaryoro/nosql_course/blob/dev/Hbase/hbase_practical_exercise2.md
    - Actions to do :
        - Extract data, transform and load on Hbase using Docker
        - Interact with data using Hbase shell
        - Package & Deploy your code